ShopVerse implementation guide

Getting the platform from zero to live on AWS at ganga888.in, step by step.

This guide takes the ShopVerse repository from an empty AWS account to a running platform. Follow the phases in order. Most phases end with a **Check** block. Don't start the next phase until those checks pass.

Phase	What you do	Result
0	Install tools and collect accounts	Laptop ready to work
1	Put the code in GitHub and replace placeholders	Your own repo with your account ID
2	Bootstrap Terraform state	S3 state bucket and lock table
3	Apply the global stack and delegate DNS	Route 53, ECR, OIDC roles, KMS, security services, SES
4	Configure GitHub	Pipelines can reach AWS with no stored keys
5	Create the dev environment	VPC and EKS for dev
6	Copy Terraform outputs into GitOps	Cluster add-ons know your real ARNs and IDs
7	Bootstrap ArgoCD	Every add-on and service syncs from Git
8	Create the Jenkins EC2 host and connect it	Images built, scanned, signed and deployed to dev
9	Deploy the web app	dev.ganga888.in serves the storefront
10	Verify dev end to end	Tests, dashboards, logs and traces checked
11	Create staging	Managed RDS, Redis and MSK, promoted images
12	Create prod and go live	ganga888.in live behind approvals and canaries
13	Day-2 operations	Nightly cost guard, backups, rotation, drills

> Conventions: `<ACCOUNT_ID>` is your 12-digit AWS account ID. `<GH_OWNER>/<GH_REPO>` is your GitHub repository. Run commands from the repository root unless a step says otherwise. Region is always `ap-south-1`.

Phase 0: Prerequisites

0.1 Accounts you need

- **An AWS account with administrator access.** Use it only for Phases 2–3 and break-glass. Prefer IAM Identity Center (SSO) over IAM users.
- **Access to the registrar of `ganga888.in`.** You will change its name servers in Phase 3.
- **A GitHub account** where you can create a repository and its environments.
- **An email inbox** for budget alerts, SNS confirmations and Alertmanager.

0.2 Tools on your laptop

Tool	Version	Used for
AWS CLI	v2	Login, kubeconfig, secrets
Terraform	≥ 1.9	Infrastructure
kubectl	1.31	Cluster access
Helm	≥ 3.16	One-time ArgoCD install
Docker + Docker Compose	recent	Running the stack locally (optional)
AWS Session Manager plugin	latest	Shell and Ansible access to the Jenkins EC2 host (no SSH)
Node.js	22	Running services locally (optional)
git, jq, yq	any recent	Scripts below
Ansible (ansible-core)	≥ 2.16	Host hardening, runbooks
k6, newman	latest	Load and end-to-end tests (optional locally; CI runs them too)

0.3 Log in to AWS

```
aws configure sso           # or: aws configure --profile shopverse-admin
export AWS_PROFILE=shopverse-admin
export AWS_REGION=ap-south-1
aws sts get-caller-identity  # note the Account value: this is <ACCOUNT_ID>
```

0.4 Optional: run everything locally first

This gives you a feel for the system before spending anything on AWS.

```
cd services
docker compose up --build -d     # Postgres, Redis, Kafka, Jaeger, 15 services, gateway on :8080
./scripts/e2e-smoke.sh           # register, browse, cart, checkout, CONFIRMED and CANCELLED paths
open http://localhost:16686      # Jaeger: one checkout trace across the services
docker compose down -v
```

Phase 1: Put the code in GitHub

1.1 Create the repository

1. Create an empty **private** repository on GitHub, for example `<GH_OWNER>/shopverse`.
2. Unzip `shopverse-aws-microservices.zip` and push it:

```
unzip shopverse-aws-microservices.zip && cd aws-microservices
git init -b main && git add . && git commit -m "ShopVerse initial import"
git remote add origin git@github.com:<GH_OWNER>/<GH_REPO>.git
git push -u origin main
```

1.2 Replace the placeholders

The repo ships with three placeholders. Replace all of them in one commit:

```
ACCOUNT_ID=<ACCOUNT_ID>
GH=<GH_OWNER>/<GH_REPO>

# 1. AWS account ID
grep -rl 111122223333 --exclude-dir=.git . | xargs sed -i "s/111122223333/${ACCOUNT_ID}/g"
# 2. Repository URL that ArgoCD pulls from
grep -rl 'https://github.com/shopverse/aws-microservices.git' --exclude-dir=.git . \
  | xargs sed -i "s#https://github.com/shopverse/aws-microservices.git#https://github.com/${GH}.git#g"
# 3. Repository allowed to assume the GitHub OIDC roles
sed -i "s#OWNER/shopverse#${GH}#g" terraform/global/terraform.tfvars

git commit -am "Set account ID and repository" && git push
```

On macOS use `sed -i ''` instead of `sed -i`.

1.3 Set your alert emails and budget

Edit `terraform/global/terraform.tfvars`:

```
budget_monthly_limit_usd = 500                      # whole account, USD
budget_alert_emails      = ["you@yourdomain.com"]
```

Check: `grep -rn 111122223333 . | wc -l` prints `0`.

Phase 2: Bootstrap Terraform state

This is run **once per account** with local state. It creates the S3 bucket `shopverse-tfstate-<ACCOUNT_ID>` (versioned and KMS-encrypted) and the DynamoDB lock table `shopverse-terraform-locks`.

```
cd terraform/bootstrap
terraform init
terraform apply
terraform output -raw backend_hcl > ../backend.hcl    # real bucket name for every other stack
cd ../.. && git add terraform/backend.hcl && git commit -m "Backend config" && git push
```

Check: `aws s3 ls | grep shopverse-tfstate` shows the bucket.

Phase 3: Global stack and DNS

The global stack holds the account-wide pieces:

- the Route 53 hosted zone for ganga888.in
- 15 ECR repositories
- the GitHub OIDC provider and its CI roles
- the cosign KMS signing key and the `shopverse-jenkins-ci` IAM policy (ECR push and signing), which the Jenkins EC2 role uses in Phase 8
- CloudTrail, GuardDuty and Security Hub
- Budgets and Cost Anomaly Detection
- the CloudFront certificate in us-east-1
- the SES email domain

3.1 Apply

```
cd terraform/global
terraform init -backend-config=../backend.hcl
terraform plan -out=global.tfplan      # read it: roughly 100+ resources, all new
terraform apply global.tfplan
terraform output route53_name_servers
```

3.2 Delegate ganga888.in to Route 53

1. At your registrar, replace the name servers of `ganga888.in` with the **four** values printed above.
2. Wait for propagation. It is usually under an hour, but can take up to 48 hours.

```
dig +short NS ganga888.in          # must list the four awsdns-* servers
```

The CloudFront certificate and the SES domain verify automatically once delegation is live. If the first apply timed out waiting for certificate validation, run `terraform apply` again after `dig` shows the new servers.

3.3 One-time console tasks

- **Confirm the SNS email subscriptions** for budget and anomaly alerts. AWS sends one email per address.
- **Activate the cost allocation tags** `Project`, `Environment` and `CostCenter` in Billing → Cost allocation tags. Per-environment budgets filter on them.
- **Request SES production access** in the SES console under Account dashboard → Request production access. Until AWS approves it, SES sends only to verified addresses (sandbox mode).

Check:

- `aws ecr describe-repositories --query 'repositories[].repositoryName'` lists the 15 `shopverse/*` repos.
- In the ACM console (us-east-1), the CloudFront certificate shows **Issued**.

Phase 4: Configure GitHub

4.1 Repository variable

Go to Settings → Secrets and variables → Actions → **Variables** and add:

- `AWS_ACCOUNT_ID` = `<ACCOUNT_ID>`

No AWS keys go into GitHub. Every workflow assumes a role through OIDC.

4.2 Environments

Under Settings → Environments, create `global`, `dev`, `staging` and `prod`.

- On **prod**, add yourself as a **required reviewer** and limit deployment branches to `main`.
- On **global**, adding a required reviewer is also recommended.

4.3 Protect main

Add a branch protection rule for `main`:

- Require a pull request.
- Require these status checks to pass: `terraform.yml` (fmt, tflint, tfsec, Checkov, plan) and the Jenkins check, once Jenkins is connected.

Check: open a pull request that only adds a comment to `terraform/envs/dev/terraform.tfvars`. The **terraform** workflow posts a plan comment on it. Close the PR without merging.

Phase 5: Create the dev environment

5.1 Give yourself kubectl access

EKS uses access entries. Add your admin or SSO role in `terraform/envs/dev/terraform.tfvars`:

```
cluster_admin_role_arns      = ["arn:aws:iam::<ACCOUNT_ID>:role/<your-admin-or-sso-role>"]
endpoint_public_access_cidrs = ["<your-public-ip>/32"]   # tighten from 0.0.0.0/0
```

Dev runs `use_managed_data = false`. That means Postgres, Redis and Kafka run as pods in the cluster, which saves about $300 a month.

5.2 Apply through a pull request

1. Create a branch, commit the tfvars change and open a PR.
2. Read the plan comment, then merge.
3. GitHub Actions applies `global`, then `dev`. Creating EKS takes roughly 15–20 minutes.

To apply by hand instead:

```
cd terraform/envs/dev
terraform init -backend-config=../../backend.hcl && terraform apply
```

5.3 Connect kubectl

```
aws eks update-kubeconfig --name shopverse-dev --region ap-south-1
kubectl get nodes        # 2 system nodes, Ready
```

Check: `terraform -chdir=terraform/envs/dev output` prints `cluster_name`, `vpc_id`, `acm_certificate_arn`, `waf_regional_acl_arn` and `irsa_role_arns`.

Phase 6: Copy Terraform outputs into GitOps

The add-ons need a few real IDs. Each place is marked `# tf-output: <name>` in `gitops/`. Find them all with `grep -rn "tf-output" gitops/envs/dev`.

```
cd terraform/envs/dev && terraform output -json > /tmp/dev-out.json && cd -
jq -r '.vpc_id.value, .vpc_cidr.value, .acm_certificate_arn.value, .waf_regional_acl_arn.value,
       .karpenter_node_role_name.value, .karpenter_interruption_queue.value' /tmp/dev-out.json
```

Placeholder	Terraform output	File
`vpcId` (AWS Load Balancer Controller)	`vpc_id`	`gitops/envs/dev/apps/platform.yaml`
`proxy-real-ip-cidr` (ingress-nginx)	`vpc_cidr`	`gitops/envs/dev/apps/platform.yaml`
`interruptionQueue` (Karpenter)	`karpenter_interruption_queue`	`gitops/envs/dev/apps/platform.yaml`
`certificate-arn` annotation	`acm_certificate_arn`	`gitops/envs/dev/platform/ingress/alb-ingress.yaml`
`wafv2-acl-arn` annotation	`waf_regional_acl_arn`	`gitops/envs/dev/platform/ingress/alb-ingress.yaml`
`role` (EC2NodeClass)	`karpenter_node_role_name`	`gitops/envs/dev/platform/karpenter/ec2nodeclass.yaml`

The IRSA role ARNs already follow the `shopverse-dev-<name>` pattern, so Phase 1.2 fixed them. Commit and push:

```
git commit -am "dev: wire Terraform outputs into GitOps" && git push
```

Check: `grep -rn "vpc-0dev0000000000000\|00000000-0000" gitops/envs/dev` prints nothing.

Phase 7: Bootstrap ArgoCD

You install ArgoCD by hand once. After that, ArgoCD manages itself and everything else from Git. On weekday mornings the nightly workflow repeats exactly this step.

```
helm repo add argo https://argoproj.github.io/argo-helm && helm repo update
helm upgrade --install argocd argo/argo-cd --namespace argocd --create-namespace \
  --version 7.8.2 \
  --set 'global.tolerations[0].key=CriticalAddonsOnly' \
  --set 'global.tolerations[0].operator=Exists' \
  --set 'configs.params.server\.insecure=true' \
  --set 'global.domain=argocd.dev.ganga888.in' \
  --wait --timeout 10m

kubectl apply -n argocd -f gitops/bootstrap/dev-root.yaml
```

7.1 Watch the platform come up

Sync waves install things in this order:

1. CRDs and operators
2. Karpenter, External Secrets, the ALB controller and ExternalDNS
3. The ingress, monitoring, logging and tracing stacks
4. The in-cluster data stores
5. The 15 services

```
kubectl -n argocd get applications -w
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d; echo
```

After about 10 minutes, ExternalDNS creates `api.dev.ganga888.in`, `argocd.dev.ganga888.in`, `grafana.dev.ganga888.in`, `kibana.dev.ganga888.in` and `jaeger.dev.ganga888.in`. All of them point at the ALB.

The services stay in `ImagePullBackOff` at this point. Their tag is `bootstrap` until Jenkins pushes real images in Phase 8. That is expected.

Check:

- Every platform application in ArgoCD is **Synced / Healthy**.
- `kubectl get externalsecrets -A` shows every entry as `SecretSynced`.
- `kubectl get nodepools` shows the Karpenter pools.
- `https://argocd.dev.ganga888.in` loads with a valid certificate.

Phase 8: Jenkins and SonarQube on EC2

Jenkins and SonarQube run together on one EC2 instance, created by `terraform/ci`:

- The instance sits in its own small CI VPC (`10.50.0.0/16`), so the nightly dev destroy never touches it.
- It is reachable only through an ALB at `https://jenkins.ganga888.in` and `https://sonarqube.ganga888.in`, and only from your IP. GitHub can reach the webhook path and nothing else.
- There is no SSH port. You and Ansible connect through SSM Session Manager.
- The instance role grants ECR push and cosign signing, so **no AWS access keys exist anywhere**.
- Jenkins home and SonarQube data sit on an encrypted EBS volume that is snapshotted every day.

8.1 Create the Jenkins host

Set your IP in `terraform/ci/terraform.tfvars` (up to 4 CIDRs):

```
admin_cidrs           = ["<your-public-ip>/32"]
instance_type         = "t3.xlarge"   # 4 vCPU / 16 GiB: Jenkins + SonarQube + parallel builds
office_hours_schedule = false         # true = stop 21:00-08:00 IST and weekends (~60% cheaper)
```

Apply it through a PR (the `apply-ci` job runs right after `global`), or by hand:

```
cd terraform/ci
terraform init -backend-config=../backend.hcl && terraform apply
terraform output        # jenkins_url, sonarqube_url, instance_id, ssm_bucket, secret_name
```

8.2 Put the GitHub token in Secrets Manager

Terraform already created `shopverse/ci/jenkins` with a random admin password and a random SonarQube DB password. Add a GitHub fine-grained token with **Contents: Read and write** on the repo. Jenkins uses it to check out code and commit image-tag bumps.

```
aws secretsmanager get-secret-value --secret-id shopverse/ci/jenkins --query SecretString --output text > /tmp/
#   edit /tmp/j.json: "GITHUB_TOKEN": "<token>"   (leave SONAR_TOKEN for 8.4)
aws secretsmanager put-secret-value --secret-id shopverse/ci/jenkins --secret-string file:///tmp/j.json && rm /
```

8.3 Configure the host with Ansible (over SSM)

On your laptop, install the Session Manager plugin and `pip install boto3 botocore`.

Fill in two files from `terraform -chdir=terraform/ci output`:

- `ansible/inventories/ci/hosts.yml`: `ansible_host` comes from `instance_id`, and `ansible_aws_ssm_bucket_name` from `ssm_bucket`.
- `ansible/inventories/ci/group_vars/all.yml`: `aws_account_id`, `github_owner` and `github_repo`.

Then run the playbook:

```
cd ansible
ansible-galaxy collection install -r requirements.yml
ansible-playbook -i inventories/ci/hosts.yml playbooks/jenkins.yml
```

The playbook does the following:

- Hardens the OS: users, SSH config, auditd, fail2ban, unattended upgrades.
- Installs Docker, node_exporter and the CloudWatch agent.
- Copies `ci/jenkins` to `/opt/jenkins` and writes the non-secret `.env`.
- Has the host itself write `.env.secrets` from Secrets Manager using its instance role. The secrets never pass through your laptop.
- Builds the Jenkins image and starts Jenkins, SonarQube and its database.

8.4 Connect SonarQube and GitHub

1. **Log in to Jenkins.** Open `https://jenkins.ganga888.in` and log in as `admin`. To get the password, run `terraform -chdir=terraform/ci output -raw jenkins_admin_password_command | bash`.
2. **Set up SonarQube.**
3. Open `https://sonarqube.ganga888.in`, log in with `admin` / `admin`, then change the password.
4. Create a token and store it in the secret as `SONAR_TOKEN` (same steps as 8.2).
5. Add the webhook `http://jenkins:8080/sonarqube-webhook/`.
6. Create the quality gate described in `ci/sonar/`.
7. **Load the token into Jenkins.** Run `ansible-playbook -i inventories/ci/hosts.yml playbooks/jenkins.yml --tags jenkins`.
8. **Add the GitHub webhook.** In GitHub, go to repo Settings → Webhooks and add `https://jenkins.ganga888.in/github-webhook/` (content type `application/json`, push and pull request events). Jenkins also scans every 5 minutes as a fallback.

8.5 First build

1. In Jenkins, open the `shopverse` job, which JCasC already created, and click **Scan Repository Now**.
2. On `main`, the first build compiles all 15 services in parallel. Each one goes through tests, the Sonar gate, `npm audit`, the Docker build, Trivy, the SBOM, the ECR push and the cosign signature.
3. The **ECR login** stage prints the instance role ARN (`shopverse-ci-jenkins`). That shows no keys are in use.
4. Jenkins then commits the new tags to `gitops/envs/dev/services/*/values.yaml` with `[skip ci]`.
5. ArgoCD deploys them within about 3 minutes.

Check:

```
aws ecr list-images --repository-name shopverse/order-service
cosign verify --key awskms:///alias/shopverse-cosign \
  <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/shopverse/order-service:<tag>
kubectl -n shop get pods            # all Running, 2/2 replicas each
```

Phase 9: Deploy the web app

1. Go to Actions → **web-deploy** → Run workflow, and choose environment `dev`.
2. The workflow builds `web/`, syncs it to `shopverse-dev-web` and invalidates CloudFront.

Later pushes that touch `web/**` deploy automatically.

Check: `https://dev.ganga888.in` lists products. In the browser network tab, `/api/v1/products` returns 200.

Phase 10: Verify dev end to end

What	How	Expect
Health	`curl https://api.dev.ganga888.in/api/v1/products`	JSON list
Functional e2e	Actions → **newman-e2e** → `dev`	All assertions pass; happy order CONFIRMED, `tok_fail` order CANCELLED
Load and autoscaling	`k6 run -e BASE_URL=https://api.dev.ganga888.in tests/k6/load.js`	p95 < 500 ms, errors < 1%; `kubectl get hpa -n shop` scales up, Karpenter adds nodes
Metrics	`https://grafana.dev.ganga888.in` → ShopVerse dashboards	Request rate, error rate and latency per service; saga funnel; Kafka lag
Logs	`https://kibana.dev.ganga888.in` → search `trace_id:<id>`	Log lines from every service in one checkout
Traces	`https://jaeger.dev.ganga888.in` → service `order-service`	One trace spanning about 11 services
Policies	`kubectl run test --image=nginx -n shop`	Rejected by Kyverno: unsigned image / registry not allowed (dev runs in Audit mode, so it shows as a policy report instead)
Secrets	`kubectl -n shop get secret order-service-db -o yaml`	Populated from Secrets Manager; nothing in Git

The dev environment is complete at this point. From now on it is destroyed at 20:00 IST and recreated at 08:00 IST on weekdays (Phase 13.1).

Phase 11: Staging

Staging uses managed data services: RDS PostgreSQL, ElastiCache Redis and MSK Kafka with SASL/SCRAM.

1. **Review the tfvars.** In `terraform/envs/staging/terraform.tfvars`, set `cluster_admin_role_arns` and tighten `endpoint_public_access_cidrs`, as in Phase 5.1.
2. **Apply** through a PR and merge. Staging applies automatically after dev. RDS and MSK take 20–30 minutes.
3. **Fill the GitOps placeholders** as in Phase 6, for `gitops/envs/staging`. There are two extra outputs: - `msk_bootstrap_brokers`: the three `:9096` brokers for kafka-exporter in `apps/platform.yaml`. - `rds_master_secret_arn`: two lines in `platform/db-init/db-init.yaml`.
4. **Bootstrap ArgoCD** as in Phase 7, with `global.domain=argocd.staging.ganga888.in` and `gitops/bootstrap/staging-root.yaml`.
5. **Check the db-init Job.** It creates one role and one database per service on RDS: `kubectl -n db-admin logs job/db-init` ends with `db-init complete`.
6. **Promote images.** Run Actions → **promote** with `from_env=dev` and `to_env=staging` for each service. Each run opens a PR that copies the tested tag. Merge the PRs. - order-service, payment-service and catalog-service roll out as canaries: 20%, then 50%, then 100%. - Watch a canary with `kubectl argo rollouts get rollout order-service -n shop -w`.
7. **Deploy and test.** Run **web-deploy** and **newman-e2e** for `staging`, then the k6 load test against `https://api.staging.ganga888.in`.

Check:

- Grafana shows Kafka lag near 0.
- `kubectl -n shop get scaledobjects` shows READY True. KEDA is authenticating to MSK.

Phase 12: Production and go-live

12.1 Before the first prod apply

- **Review** `terraform/envs/prod/terraform.tfvars`**.** It sets 3 AZs, 3 NAT gateways, a Multi-AZ RDS on `db.r6g.large` with PITR, Redis with a primary and 2 replicas, and 3 MSK brokers on `kafka.m5.large`.
- **Check service quotas** in ap-south-1: EC2 vCPUs (On-Demand and Spot), Elastic IPs (at least 3) and VPCs.
- **Price it.** Expect roughly $1,800–3,000 a month before Savings Plans. Raise `budget_monthly_limit_usd` to match.

12.2 Apply prod

1. Merge the PR. The prod job pauses at the **prod** environment until a required reviewer approves it.
2. Fill the placeholders in `gitops/envs/prod`, as in Phase 11.3.
3. Bootstrap ArgoCD with `gitops/bootstrap/prod-root.yaml` and `global.domain=argocd.ganga888.in`.

12.3 Set the real secrets

Terraform creates these as placeholders and never overwrites them afterwards:

```
aws secretsmanager put-secret-value --secret-id shopverse/prod/payment/gateway \
  --secret-string '{"api_key":"<real payment gateway key>"}'
aws secretsmanager put-secret-value --secret-id shopverse/prod/monitoring/alertmanager \
  --secret-string '{"slack_webhook_url":"https://hooks.slack.com/services/...","smtp_username":"...","smtp_pass
```

External Secrets refreshes them within an hour. To refresh at once, run `kubectl annotate externalsecret --all -n shop force-sync=$(date +%s) --overwrite`.

12.4 Promote and go live

1. Promote every service from `staging` to `prod` with the **promote** workflow. Prod PRs need your approval.
2. Run **web-deploy** with `prod`.
3. Run **newman-e2e** with `prod`, using the test card `tok_fail` only for the cancel path.

12.5 Go-live checklist

☐ `https://ganga888.in` and `https://www.ganga888.in` serve the storefront through CloudFront. `https://api.ganga888.in` answers.
☐ Every ArgoCD app is Synced and Healthy. No Kyverno policy violations (Enforce mode in prod).
☐ RDS is Multi-AZ, automated backups are on, and AWS Backup has a cross-region copy job.
☐ `velero get schedules` shows the daily and hourly schedules. The first backup has completed.
☐ A test alert reaches Slack and email (`amtool` or a firing test rule).
☐ Budgets and anomaly alerts are confirmed. GuardDuty and Security Hub show no high findings.
☐ The SES domain is verified and production access is granted. A test order email arrives.
☐ A rollback was rehearsed: revert the last tag commit and watch ArgoCD roll back.

Phase 13: Day-2 operations

13.1 Nightly cost guard for dev

`.github/workflows/nightly-dev.yml` runs on this schedule:

- **20:00 IST daily:** scales ArgoCD to zero, deletes the ALB ingresses, NodePools and PVCs, then destroys EKS, the NAT gateway and the Elastic IPs.
- **08:00 IST, Monday to Friday:** recreates the cluster and repeats Phase 7 automatically.

Run either job on demand from Actions with `workflow_dispatch`.

13.2 Backups and disaster recovery

- **Targets:** RTO 1 hour and RPO 15 minutes in prod. See `docs/runbooks/dr-restore.md`.
- **Quarterly drill:** `ansible-playbook ansible/playbooks/runbooks/rds-restore-drill.yml -e env=prod`. It restores the latest snapshot to a temporary instance, checks it and deletes it.

13.3 Look after the Jenkins host

- **Shell access:** `aws ssm start-session --target <instance_id>`. Port 22 is closed.
- **OS patches and config drift:** re-run `ansible-playbook -i inventories/ci/hosts.yml playbooks/jenkins.yml` monthly. Unattended upgrades cover security patches in between.
- **Backups:** the data volume (Jenkins home and SonarQube) is snapshotted daily at 02:00 IST and kept 14 days. To restore, create a volume from a snapshot, swap it in for the data volume and reboot.
- **New AMI:** `terraform -chdir=terraform/ci apply -replace=aws_instance.jenkins`, then re-run the playbook. The data volume is kept, and is protected from `destroy`.
- **Cost:** about $150 a month. Turn on `office_hours_schedule` to stop it outside working hours.

13.4 Rotation

- **Jenkins AWS access:** nothing to rotate. The EC2 instance role's credentials are short-lived and refreshed by AWS.
- **Jenkins tokens (GitHub, SonarQube, admin password):** update `shopverse/ci/jenkins` in Secrets Manager, then re-run the Jenkins playbook with `--tags jenkins`.
- **RDS master password:** rotated by AWS automatically.
- **Service DB passwords and JWT keys:** see `docs/runbooks/secret-rotation.md`.

13.5 Everyday changes

I want to…	Do this
Ship a code change	PR → Jenkins checks → merge → auto-deployed to dev → promote PR to staging → promote PR to prod
Roll back	`git revert` the tag commit, or `kubectl argo rollouts undo <svc> -n shop`. See `docs/runbooks/rollback.md`
Change infrastructure	Edit Terraform → PR (plan comment) → merge → dev/staging auto-apply, prod after approval
Add a 16th service	`docs/runbooks/onboarding-new-service.md`
Scale	`docs/runbooks/scaling.md` (HPA, KEDA lag thresholds, Karpenter limits)
Handle an incident	`docs/runbooks/incident-response.md`
Cut cost	Kubecost dashboard + `docs/runbooks/cost-governance.md`; `terraform destroy` an environment you don't need

Troubleshooting

Symptom	Likely cause	Fix
CloudFront or ALB certificate stuck in *Pending validation*	DNS not yet delegated	Finish Phase 3.2; `dig NS ganga888.in` must show awsdns servers, then re-apply
GitHub workflow fails at "Configure AWS credentials"	`AWS_ACCOUNT_ID` variable missing, or the repo name in `github_repo` doesn't match	Fix Phase 4.1 / 1.2 and re-apply `terraform/global` as admin
`kubectl` returns *Unauthorized*	Your role isn't in `cluster_admin_role_arns`	Add it (Phase 5.1) and apply
ArgoCD apps *OutOfSync* with "repository not found"	Repo URL placeholder or private repo without credentials	Fix Phase 1.2; for a private repo add it in ArgoCD → Settings → Repositories (GitHub App or deploy key)
ExternalSecret *SecretSyncedError*	IRSA role ARN or account ID wrong	`kubectl describe externalsecret <name> -n <ns>`; check the role ARN annotations match `terraform output irsa_role_arns`
No ALB created	`vpcId` / certificate / WAF placeholders still dummy	Phase 6; `kubectl -n kube-system logs deploy/aws-load-balancer-controller`
Pods *Pending* forever	Karpenter node role or interruption queue placeholder wrong	Phase 6; `kubectl -n kube-system logs deploy/karpenter`
Services *ImagePullBackOff* with tag `bootstrap`	Jenkins hasn't pushed yet	Phase 8.5
`jenkins.ganga888.in` returns 403 "Forbidden"	Your IP isn't in `admin_cidrs` (it changed?)	Update `terraform/ci/terraform.tfvars` and apply
ECR login fails in Jenkins with "Unable to locate credentials"	Container can't reach IMDS, or `AWS_AUTH_MODE` isn't `instance-profile`	Instance metadata hop limit must be 2 (set by Terraform); check `/opt/jenkins/.env`
Ansible can't connect to the Jenkins host	Session Manager plugin missing, or wrong instance ID / bucket	`aws ssm start-session --target <id>` must work first; recheck `inventories/ci/hosts.yml`
Pods rejected by Kyverno in staging/prod	Image not signed by the cosign KMS key	Build through Jenkins; never push by hand
Orders stay PENDING	Kafka unreachable or consumer lag	Grafana Kafka lag dashboard; `kubectl -n shop logs deploy/inventory-service`; check the `.dlq` topics
Nightly destroy leaves the VPC behind	Leftover ALB, ENI or EBS from Kubernetes	Re-run the destroy job; it sweeps tagged resources (see `docs/runbooks/nightly-destroy-and-recreate.md`)